01 / SCORING

RFM 기준일과 점수

기준일: 2013-03-13 00:00:00 UTC (한국시간 09:00). 데이터의 마지막 관측일 다음 날을 공통 기준으로 사용합니다. 현재 날짜로 계산하면 역사 데이터의 최근성을 비교할 수 없습니다.

요소	사용자별 산식	점수 방향
R 최근성	(기준시각 - 마지막 timestamp) / 86,400	작을수록 높음
F 빈도	count(distinct timestamp)	클수록 높음
M 학습량	sum(session_seen)	클수록 높음

사용자 분포의 20·40·60·80백분위로 5개 구간을 만들고, R만 점수 방향을 뒤집었습니다. 같은 경계값을 가진 사용자는 같은 점수로 유지하므로 각 점수의 인원이 정확히 20%씩은 아닙니다.

점수	R 경과 일수	F 세션 수	M 어휘 노출
5	≤ 1.263963	≥ 11	≥ 277
4	1.263963 초과~3.193275 이하	5~10	119~276
3	3.193275 초과~5.761671 이하	3~4	56~118
2	5.761671 초과~8.499238 이하	2	26~55
1	> 8.499238	1	≤ 25

R의 표시값은 소수 6자리로 반올림했습니다. 실제 계산은 반올림 전 경계값을 사용합니다. F=1인 33,874명은 모두 F=1점을 받습니다. RFM_code = 100×R점수 + 10×F점수 + M점수는 식별 코드이며, 숫자 크기 자체를 종합 가치로 해석하지 않습니다.

계산 단위를 구분해야 하는 이유

한 세션에 여러 단어 행이 있으므로 행 수를 세션 수로 세지 않습니다. F는 사용자별 서로 다른 timestamp의 수인 세션 대리값입니다. M은 단어 노출 합계이며 고유 문항 수·학습 시간·매출이 아닙니다. 과거 누적값인 history_seen은 M에 합산하지 않습니다.

delta는 같은 단어의 이전 학습 이후 시간입니다. 사용자 접속 간격과 구분해야 하며, 전체 delta 분산을 곧바로 학습 습관의 규칙성으로 해석하지 않습니다.

02 / SEGMENT RULES

7개 세그먼트 규칙

아래 R·F·M은 원래 값이 아닌 1~5점 스코어입니다. **위에서 아래 순서로 처음 충족하는 그룹**에 배정합니다. 125개 점수 조합이 모두 분류되는지 확인했고 사용자마다 한 그룹만 부여했습니다.

세그먼트	규칙	행동 해석
핵심 학습형	R≥4, F≥4, M≥4	최근 활동·반복 빈도·학습량이 모두 높음
반복 학습형	R≥3, F≥4; 핵심군 제외	반복 경험은 있으나 최근성 또는 학습량이 핵심군보다 낮음
집중 성장형	R≥4, F≤3, M≥3	최근 학습량은 있으나 반복 빈도가 낮아 습관화 여지가 있음
최근 저강도형	R≥4, F≤3, M≤2	최근 활동했지만 학습 빈도와 양이 적음
관심 유지형	R=3, F≤3	최근성이 중간 수준이며 반복 습관이 부족함
우선 복귀형	R≤2, F≥3 또는 M≥3	최근 활동은 뜸하지만 기존 학습 경험이 상대적으로 있음
저활동·휴면형	R≤2, F≤2, M≤2	최근 활동과 관측된 학습 경험이 모두 적음

분류 예시

RFM 555는 핵심 학습형, 432는 최근 저강도형, 133은 우선 복귀형에 해당합니다. 단순 점수 합계 대신 최근성과 반복 경험의 조합을 사용해 액션을 다르게 설계합니다.

> 집중 성장형은 성장 가능성을 나타내는 운영 이름입니다. 실제 성과 상승을 확인했다는 뜻은 아닙니다. 가입일이 없어 최근 저강도형을 신규 사용자로, 약 12일간 관측한 휴면형을 영구 이탈 고객으로 단정할 수 없습니다.

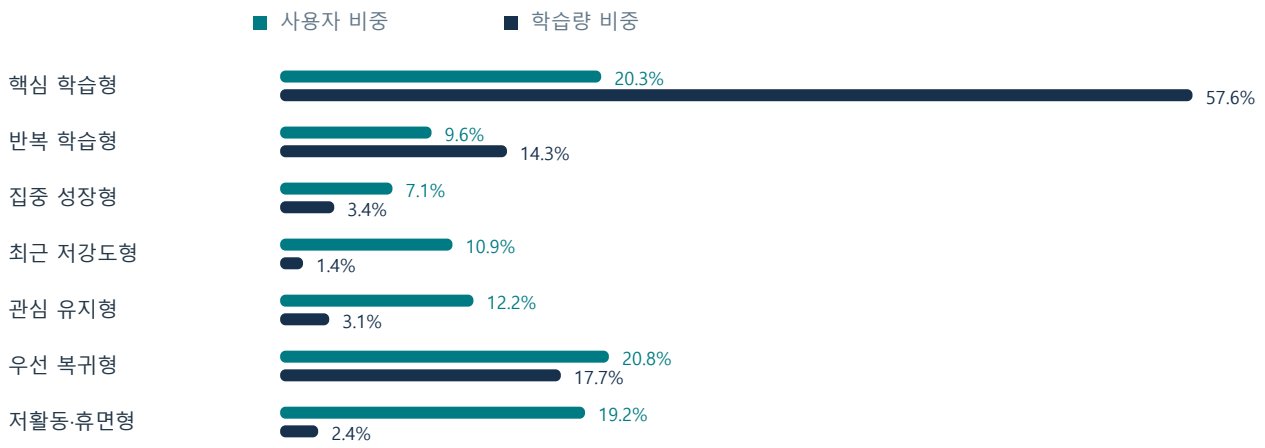
마지막 관측일 가까이에 처음 등장한 사용자는 F와 M을 쌓을 기회가 짧습니다. 운영 데이터에서는 가입·최초 관측 경과일과 학습 언어를 함께 확인하고, 코호트별로 규칙을 재검증하는 것이 필요합니다.

03 / SEGMENT SUMMARY

사용자 규모와 학습량 비중

사용자 비중의 분모는 **115,222명**, 학습량 비중의 분모는 **23,364,844회**의 어휘 노출입니다. 매출 정보가 없으므로 매출 비중은 모든 그룹에서 산출 불가입니다.

세그먼트	사용자 수	사용자 비중	매출 비중	학습량 비중
핵심 학습형	23,353	20.27%	산출 불가	57.60%
반복 학습형	11,014	9.56%	산출 불가	14.32%
집중 성장형	8,171	7.09%	산출 불가	3.42%
최근 저강도형	12,535	10.88%	산출 불가	1.45%
관심 유지형	14,060	12.20%	산출 불가	3.10%
우선 복귀형	23,912	20.75%	산출 불가	17.72%
저활동·휴면형	22,177	19.25%	산출 불가	2.40%
합계	115,222	100.00%	산출 불가	100.00%



집계에서 확인한 우선순위

핵심·반복 학습형 34,367명은 사용자 29.83%지만 학습량 71.92%를 차지합니다. 기존 학습 활동을 유지하는 대상 풀입니다. **우선 복귀형 23,912명**은 사용자 20.75%, 학습량 17.72%로 기존 학습 이력을 활용한 복귀 실험의 우선 대상입니다.

최근 저강도형 12,535명은 짧은 복습을 통해 다음 학습을 유도하는 대상으로 분리할 수 있습니다. 비중 표시값은 반올림 때문에 합계와 조금 다를 수 있습니다.

03 / FORWARD VALIDATION

이후 재학습과 연결되는가

전체 기간으로 만든 세그먼트와 같은 기간의 활동을 비교하면 최근성과 결과가 겹칩니다. 별도 검증에서는 **3월 1~7일의 데이터만으로** 점수 경계를 다시 계산하고, **3월 8~11일 중 한 번 이상 재학습**했는지 확인했습니다.

검증 모집단은 앞 기간에 관측된 **82,467명**입니다. 이 표의 사용자 수와 기준 경계는 전체 기간 요약표와 다릅니다.

세그먼트	검증 대상	이후 재학습	4일 재학습률
핵심 학습형	16,004	11,253	70.31%
반복 학습형	7,051	3,365	47.72%
집중 성장형	6,669	2,835	42.51%
최근 저강도형	9,120	2,797	30.67%
관심 유지형	10,636	2,677	25.17%
우선 복귀형	16,849	3,961	23.51%
저활동·휴면형	16,138	2,352	14.57%

핵심 학습형 70.31%, 최근 저강도형 30.67%, 우선 복귀형 23.51%, 저활동·휴면형 14.57%로 이후 활동에 차이가 관측됐습니다. 이 지표는 D7 리텐션 또는 주간 4일 지속 학습률이 아니며, 마케팅의 인과 효과도 아닙니다.

노션 NSM의 별도 주간 집계

집계 기간	주간 활성 사용자	4일 이상 학습	지속 학습 비율
3월 5~11일	83,325명	9,838명	11.81%

첫날과 마지막 날은 부분 관측이므로 NSM은 온전히 관측된 7일을 사용했습니다. 위 지속 학습 비율의 분모는 주간 활성 사용자입니다. 새 A/B 실험의 효과를 평가할 때는 배정된 전체 사용자를 분모로 쓰는 지표를 함께 사용합니다.

04 / MARKETING ACTIONS

대상별 액션과 움직일 지표

대상 규모는 전체 기간 스냅샷의 **분석상 대상 풀**입니다. 실제 운영에는 계정 매핑, 도달 가능 여부, 수신 동의 및 중복 캠페인 제외 조건을 적용합니다. 지표 방향은 검증할 가설입니다.

세그먼트	대상 풀	제안 액션	움직일 지표
핵심 학습형	23,353명	개인 진행 리포트와 주간 4일 학습 미션	주간 지속 학습 사용자 수 ↑
반복 학습형	11,014명	오답 단어 복습 큐와 학습 시간대 리마인더	다음날 재학습률 ↑
집중 성장형	8,171명	집중 학습을 여러 날의 짧은 복습으로 연결하는 3일 미션	활동일 수·재학습률 ↑
최근 저강도형	12,535명	최근 학습 단어로 시작하는 3분 복습 진입점	48시간 내 재학습률 ↑
관심 유지형	14,060명	마지막 학습 단어를 활용한 개인화 복습 안내	7일 내 재활성률 ↑
우선 복귀형	23,912명	전에 배운 단어 3개로 시작하는 복귀 안내	7일 내 재활성률 ↑ 복귀 후 재학습률 ↑
저활동·휴면형	22,177명	기초 복습으로 바로 시작하는 간단한 복귀 동선	재활성률·이후 반복 활동 ↑

실험 우선순위와 분모

우선 복귀형과 최근 저강도형부터 실험합니다. 세그먼트 안에서 사용자 단위로 실험군·대조군을 무작위 배정하고 학습 언어와 관측 경과일을 균형 있게 맞춥니다. 표본 수는 기준 재학습률과 검출하려는 차이를 정한 뒤 계산합니다.

7일 내 재활성률 = 7일 내 한 번 이상 학습한 배정 사용자 / 전체 배정 사용자.
개입 후 4일 이상 학습률 = 7일 중 4일 이상 학습한 배정 사용자 / 전체 배정 사용자.
다음날 재학습률 = 다음날도 활동한 사용자-활동일 / 다음날까지 관측 가능한 사용자-활동일.

학습 성과는 sum(session_correct) / sum(session_seen)로 함께 관찰합니다. 언어·어휘 난이도를 고려하고, 반복 사용자-활동일의 통계 추론은 사용자 단위로 묶어 처리합니다.

05 / K-MEANS

4개 군집으로 비교

같은 115,222명의 연속형 R·F·M을 log1p 변환 후 표준화했습니다. NumPy 구현의 Lloyd K-means와 k-means++ 초기화를 사용하고, K별로 20회 초기화 중 군집 내 제곱거리 합이 최소인 결과를 선택했습니다.

군집 수 K	실루엣 점수	군집 내 제곱거리 합
3	0.3583	131,061.13
4	0.3734	95,114.18
5	0.3443	79,608.69

실루엣은 같은 무작위 표본 2,000명에서 계산했습니다. 비교한 K=3~5 범위에서 점수가 가장 높은 **K=4**를 채택했습니다. 전체 모집단의 최적 K를 확정하는 결론은 아닙니다.

군집	사용자 수	R 중앙값 일	F 중앙값	M 중앙값	행동 해석
C1	38,219	7.86	1	28	비활성·저학습량
C2	25,225	1.38	2	54	최근 활동·낮은 반복 빈도
C3	29,189	6.24	5	153	활동이 뜸해진 중간 빈도·학습량
C4	22,589	1.23	18	471	최근 활동·고빈도·고학습량

군집에서 드러난 구조

C4는 최근성과 반복 경험이 높은 핵심 활동군입니다. C2는 최근 활동했지만 반복 빈도가 낮은 군집입니다. C3는 기존 학습 경험이 있으나 활동이 뜸해진 사용자가 많고, C1은 비활성·저학습량 사용자를 포괄합니다.

F와 M의 피어슨 상관은 0.9085로 높습니다. 행동 유형 연구를 확장할 때는 활동일 수, 동일 단어별 복습 간격, 세션 간격의 규칙성 등을 추가한 별도 분석이 필요합니다.

05 / SEGMENT COMPARISON

규칙형 분류와 군집의 연결

다음 값은 **각 RFM 세그먼트 안에서** K-means 군집이 차지하는 비중입니다. 행의 합이 약 100%이며, 반올림으로 약간의 차이가 있습니다.

RFM 세그먼트	C1	C2	C3	C4
핵심 학습형	0.0%	9.9%	6.5%	83.6%
반복 학습형	0.3%	16.1%	60.0%	23.6%
집중 성장형	0.0%	96.7%	3.3%	0.0%
최근 저강도형	5.9%	94.1%	0.0%	0.0%
관심 유지형	64.5%	10.2%	25.3%	0.0%
우선 복귀형	25.9%	0.0%	72.1%	2.0%
저활동·휴면형	100.0%	0.0%	0.0%	0.0%

ARI = 0.4492. 두 분류의 일부 구분은 겹치지만 군집 수와 경계가 다릅니다. 이 수치는 정확도 44.92%를 의미하지 않습니다.

운영에 반영할 해석

핵심 학습형의 **83.6%**는 C4, 우선 복귀형의 **72.1%**는 C3에 속합니다. 집중 성장형과 최근 저강도형은 각각 **96.7%, 94.1%**가 C2에 속해 큰 행동 유형은 비슷하지만 학습량에 따라 액션을 다르게 설계할 수 있습니다.

관심 유지형은 C1과 C3로 나뉩니다. 같은 규칙형 세그먼트 안에서도 군집을 보조 태그로 붙여 액션 효과가 달라지는지 확인할 수 있습니다.

캠페인 대상 선정에는 7개 RFM 규칙, 내부 행동 차이 확인에는 4개 K-means 군집을 제안합니다. 두 방식은 같은 R·F·M을 입력했으므로 분류가 겹친다는 사실 자체가 독립적인 타당성 검증은 아닙니다. 기간 분리 검증과 이후 무작위 실험으로 평가합니다.

APPENDIX A / REPRODUCTION

코드 실행 안내와 출처

다음 부록은 이번 집계에 실제 사용한 **analyze_rfm.py 전체 280줄**입니다. 원본 .py 파일도 PDF의 첨부 파일에 포함했습니다. PDF에서 복사하면 줄바꿈이 달라질 수 있으므로 실행에는 첨부 파일 또는 작업 폴더의 원본 코드를 사용합니다.

항목	설정
재현 환경	Python 3.12.14, pandas 3.0.1, NumPy 2.3.5
원본 파일	settles.acl16.learning_traces.13m.csv.gz
파일 배치	analyze_rfm.py와 원본 CSV.gz를 같은 폴더에 배치
패키지 설치	python -m pip install pandas==3.0.1 numpy==2.3.5
실행	python analyze_rfm.py
재실행 캐시	sessions.pkl 및 quality.json이 있으면 원본 집계를 재사용

데이터 파일을 변경했다면 기존 sessions.pkl과 quality.json을 다른 곳에 보관한 뒤 실행해 새 원본을 다시 집계해야 합니다. 첫 실행은 원본을 두 번 읽어 해시 후보의 실제 값까지 비교하며 완전 중복만 제거합니다.

주요 결과 파일

results.json: 원래 경계값과 분석 결과
user_rfm.csv: 사용자별 점수·세그먼트·군집
segment_summary.csv: 그룹별 인원·비중·주간 지표
forward_validation.csv: 이후 4일 재학습률
cluster_summary.csv 및 segment_cluster_*.csv: 군집·교차표

출처와 품질 확인

3팀 프로젝트 노션

공식 원본: Settles & Meeder 연구 데이터

Duolingo 변수 설명

K-means 참고 문서

ARI 참고 문서

원본 12,854,226행 - 완전 중복 81행 = 12,854,145행. 고유 사용자 115,222명. 원본 MD5는 공식 메타데이터와 일치했습니다. 세그먼트 인원·어휘 노출 합계, 125개 점수 조합의 전체 분류, 소규모 K-means 예제 및 ARI 레이블 치환 불변성을 확인했습니다.

가입일·결제·메시지 반응이 없는 역사 데이터입니다. 장기 이탈, 매출 기여, 마케팅 액션의 인과 효과는 이 결과로 확정하지 않습니다. 세부 결과와 코드의 재현성이 이번 문서의 범위입니다.

APPENDIX B / SOURCE CODE 01

전체 파이썬 코드

analyze_rfm.py | 원본 1~52행 | 줄 번호가 없는 점 표시는 긴 줄의 이어지는 부분입니다.

```python
"""Reproduce the Duolingo learning RFM analysis using pandas and NumPy.

M measures lexeme exposures, not money, elapsed time, or unique exercises.
All calendar boundaries use UTC. The public data are a learning trace extract.
"""
import os
os.environ.setdefault('OPENBLAS_NUM_THREADS', '1')
os.environ.setdefault('OMP_NUM_THREADS', '1')

from pathlib import Path
import hashlib
import json
import time
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent
SOURCE = ROOT / 'settles.acl16.learning_traces.13m.csv.gz'
COLS = ['p_recall', 'timestamp', 'delta', 'user_id', 'learning_language',
        'ui_language', 'lexeme_id', 'lexeme_string', 'history_seen',
        'history_correct', 'session_seen', 'session_correct']
DTYPES = {c: 'int64' for c in ['timestamp', 'delta', 'history_seen',
          'history_correct', 'session_seen', 'session_correct']}
DTYPES.update({c: 'object' for c in ['user_id', 'learning_language',
              'ui_language', 'lexeme_id', 'lexeme_string']})
DTYPES['p_recall'] = 'float64'
SEGMENTS = ['핵심 학습형', '반복 학습형', '집중 성장형', '최근 저강도형',
            '관심 유지형', '우선 복귀형', '저활동·휴면형']

def epoch(s):
    return int(pd.Timestamp(s, tz='UTC').timestamp())

def aggregate_source():
    started = time.time()
    chunks, hashes = [], []
    n, missing, invalid = 0, 0, 0
    for i, c in enumerate(pd.read_csv(SOURCE, dtype=DTYPES, chunksize=250_000)):
        missing += int(c.isna().any(axis=1).sum())
        valid = ((c.session_seen > 0) & (c.session_correct >= 0) &
                 (c.session_correct <= c.session_seen) & (c.timestamp > 0) &
                 (c.delta >= 0) & c.p_recall.between(0, 1))
        invalid += int((~valid).sum())
        hashes.append(pd.util.hash_pandas_object(c[COLS], index=False).to_numpy())
        g = c.groupby(['user_id', 'timestamp'], sort=False).agg(
            exposures=('session_seen', 'sum'), correct=('session_correct', 'sum'),
            traces=('session_seen', 'size'))
        chunks.append(g)
        n += len(c)
        if (i + 1) % 8 == 0:
            print(f'Read {n:,} rows; {time.time()-started:.1f}s', flush=True)
    assert missing == 0 and invalid == 0, (missing, invalid)
    h = np.concatenate(hashes)
```

APPENDIX B / SOURCE CODE 02

전체 파이썬 코드 · 계속

analyze_rfm.py | 원본 53~104행 | 줄 번호가 없는 점 표시는 긴 줄의 이어지는 부분입니다.

```
    order = np.argsort(h, kind='stable')
    same = h[order[1:]] == h[order[:-1]]
    # Hashes locate candidates only. Full values below verify exact duplicates.
    candidates = np.unique(np.r_[order[:-1][same], order[1:][same]])
    print(f'Checking {len(candidates)} candidate rows for exact duplicates', flush=True)
    candidate_rows, offset = [], 0
    for c in pd.read_csv(SOURCE, dtype=DTYPES, chunksize=250_000):
        ids = candidates[(candidates >= offset) & (candidates < offset+len(c))]
        if len(ids):
            t = c.iloc[ids-offset].copy()
            t.index = ids
            candidate_rows.append(t)
        offset += len(c)
    duplicates = pd.DataFrame(columns=COLS)
    if candidate_rows:
        candidates_df = pd.concat(candidate_rows).sort_index()
        duplicates = candidates_df[candidates_df.duplicated(COLS, keep='first')]
    sessions = pd.concat(chunks).groupby(level=[0, 1], sort=False).sum()
    if len(duplicates):
        adjustments = duplicates.groupby(['user_id', 'timestamp']).agg(
            exposures=('session_seen', 'sum'), correct=('session_correct', 'sum'),
            traces=('session_seen', 'size'))
        sessions.loc[adjustments.index] -= adjustments
    sessions = sessions.reset_index().sort_values(['user_id', 'timestamp'])
    assert (sessions.exposures > 0).all()
    assert int(sessions.traces.sum()) == n-len(duplicates)
    sessions.to_pickle(ROOT / 'sessions.pkl')
    meta = {'raw_rows': n, 'exact_duplicates_removed': len(duplicates),
            'clean_rows': n-len(duplicates), 'missing_rows': missing,
            'invalid_rows': invalid, 'unique_users': int(sessions.user_id.nunique()),
            'session_proxy_count': len(sessions),
            'min_timestamp': int(sessions.timestamp.min()),
            'max_timestamp': int(sessions.timestamp.max()),
            'min_utc': str(pd.to_datetime(sessions.timestamp.min(), unit='s', utc=True)),
            'max_utc': str(pd.to_datetime(sessions.timestamp.max(), unit='s', utc=True)),
            'sha256': hashlib.file_digest(SOURCE.open('rb'), 'sha256').hexdigest()}
    (ROOT / 'quality.json').write_text(json.dumps(meta, indent=2), encoding='utf-8')
    return sessions, meta

def quintile(values, reverse=False):
    cuts = np.quantile(values, [0.2, 0.4, 0.6, 0.8], method='linear')
    # Right-closed intervals: a tie at a boundary stays in the lower raw bin.
    raw_score = 1 + np.searchsorted(cuts, values, side='left')
    return (6-raw_score if reverse else raw_score), cuts

def assign_segment(u):
    r, f, m = u.R_score, u.F_score, u.M_score
    masks = [
        (r >= 4) & (f >= 4) & (m >= 4),
        (r >= 3) & (f >= 4),
        (r >= 4) & (f <= 3) & (m >= 3),
        (r >= 4) & (f <= 3) & (m <= 2),
```

APPENDIX B / SOURCE CODE 03

전체 파이썬 코드 · 계속

analyze_rfm.py | 원본 105~156행 | 줄 번호가 없는 점 표시는 긴 줄의 이어지는 부분입니다.

```
        (r == 3) & (f <= 3),
        (r <= 2) & ((f >= 3) | (m >= 3)),
        (r <= 2) & (f <= 2) & (m <= 2)]
    u['segment'] = np.select(masks, SEGMENTS, default='UNASSIGNED')
    assert (u.segment != 'UNASSIGNED').all()
    return u

def user_snapshot(s, start, cutoff):
    p = s[(s.timestamp >= start) & (s.timestamp < cutoff)].copy()
    p['day'] = p.timestamp // 86400
    u = p.groupby('user_id').agg(last_timestamp=('timestamp', 'max'),
        first_timestamp=('timestamp', 'min'), F=('timestamp', 'size'),
        M=('exposures', 'sum'), correct=('correct', 'sum'),
        active_days=('day', 'nunique'))
    u['R'] = (cutoff-u.last_timestamp)/86400
    u['recall_rate'] = u.correct/u.M
    thresholds = {}
    for name in ['R', 'F', 'M']:
        u[name+'_score'], cuts = quintile(u[name].to_numpy(), name == 'R')
        thresholds[name] = cuts.tolist()
    u['RFM_code'] = (100*u.R_score + 10*u.F_score + u.M_score).astype(int)
    return assign_segment(u), thresholds

def summary(u):
    g = u.groupby('segment').agg(users=('R', 'size'), exposures=('M', 'sum'),
        correct=('correct', 'sum'), R_median=('R', 'median'),
        F_median=('F', 'median'), M_median=('M', 'median'),
        active_days_median=('active_days', 'median'))
    g = g.reindex(SEGMENTS)
    g['user_pct'] = g.users/len(u)*100
    g['exposure_pct'] = g.exposures/u.M.sum()*100
    g['weighted_recall_pct'] = g.correct/g.exposures*100
    g['revenue_pct'] = np.nan
    assert int(g.users.sum()) == len(u)
    assert int(g.exposures.sum()) == int(u.M.sum())
    return g

def kmeans(x, k, seed, n_init=20):
    rng = np.random.default_rng(seed)
    best = None
    for _ in range(n_init):
        centers = [x[rng.integers(len(x))]]
        while len(centers) < k:
            d = np.min(np.stack([np.sum((x-c)**2, axis=1) for c in centers]), axis=0)
            centers.append(x[rng.choice(len(x), p=d/d.sum())])
        centers = np.array(centers)
        for iteration in range(200):
            d = np.stack([np.sum((x-c)**2, axis=1) for c in centers], axis=1)
            labels = d.argmin(axis=1)
            new = np.array([x[labels == j].mean(axis=0) if np.any(labels == j)
                            else x[np.argmax(d.min(axis=1))] for j in range(k)])
            shift = float(np.sum((new-centers)**2))
```

APPENDIX B / SOURCE CODE 04

전체 파이썬 코드 · 계속

analyze_rfm.py | 원본 157~208행 | 줄 번호가 없는 점 표시는 긴 줄의 이어지는 부분입니다.

```
            centers = new
            if shift < 1e-8:
                break
        d = np.stack([np.sum((x-c)**2, axis=1) for c in centers], axis=1)
        labels = d.argmin(axis=1)
        inertia = float(d[np.arange(len(x)), labels].sum())
        if best is None or inertia < best[0]:
            best = (inertia, labels, centers, iteration+1)
    return best

def silhouette(x, labels, indices):
    a, y = x[indices], labels[indices]
    sq = np.sum(a*a, axis=1)
    distances = np.sqrt(np.maximum(sq[:, None]+sq[None, :]-2*a@a.T, 0))
    np.fill_diagonal(distances, 0)
    intra = np.zeros(len(a))
    inter = np.full(len(a), np.inf)
    counts = np.bincount(y)
    for j in np.unique(y):
        mask = y == j
        mean_distance = distances[:, mask].sum(axis=1)/counts[j]
        inter[~mask] = np.minimum(inter[~mask], mean_distance[~mask])
        if counts[j] > 1:
            intra[mask] = distances[mask][:, mask].sum(axis=1)/(counts[j]-1)
    score = (inter-intra)/np.maximum(intra, inter)
    score[counts[y] == 1] = 0
    return float(score.mean())

def ari(a, b):
    c = pd.crosstab(a, b).to_numpy(dtype=float)
    choose2 = lambda v: np.sum(v*(v-1)/2)
    n = c.sum()
    both, row, col = choose2(c), choose2(c.sum(axis=1)), choose2(c.sum(axis=0))
    expected = row*col/(n*(n-1)/2)
    maximum = (row+col)/2
    return float((both-expected)/(maximum-expected)) if maximum != expected else 1.0

def main():
    if (ROOT/'sessions.pkl').exists():
        s = pd.read_pickle(ROOT/'sessions.pkl')
        meta = json.loads((ROOT/'quality.json').read_text(encoding='utf-8'))
    else:
        s, meta = aggregate_source()
    T = epoch('2013-03-13')
    u, thresholds = user_snapshot(s, int(s.timestamp.min()), T)
    # Last completely observed calendar week; exclude both partial boundary dates.
    w = s[(s.timestamp >= epoch('2013-03-05')) & (s.timestamp < epoch('2013-03-12'))].copy()
    w['day'] = w.timestamp//86400
    days = w.groupby('user_id').day.nunique()
    u['week_active_days'] = days.reindex(u.index, fill_value=0)
    u['week_sustained'] = u.week_active_days >= 4
    pair = w[['user_id', 'day']].drop_duplicates().sort_values(['user_id', 'day'])
```

APPENDIX B / SOURCE CODE 05

전체 파이썬 코드 · 계속

analyze_rfm.py | 원본 209~257행 | 줄 번호가 없는 점 표시는 긴 줄의 이어지는 부분입니다.

```
    pair['next_day'] = pair.groupby('user_id').day.shift(-1)
    eligible = pair[pair.day < epoch('2013-03-11')//86400].copy()
    eligible['return_next_day'] = eligible.next_day == eligible.day+1
    d1 = eligible.groupby('user_id').agg(d1_eligible=('day', 'size'),
        d1_return=('return_next_day', 'sum'))
    u = u.join(d1).fillna({'d1_eligible': 0, 'd1_return': 0})
    g = summary(u)
    wg = u.groupby('segment').agg(weekly_active=('week_active_days', lambda x: int((x >= 1).sum()))
    ,
        weekly_sustained=('week_sustained', 'sum'), d1_eligible=('d1_eligible', 'sum'),
        d1_return=('d1_return', 'sum'))
    g = g.join(wg)
    g['weekly_sustained_pct_among_weekly_active'] = g.weekly_sustained/g.weekly_active*100
    g['next_day_return_pct'] = g.d1_return/g.d1_eligible*100
    g.to_csv(ROOT/'segment_summary.csv', encoding='utf-8-sig')
    # Short forward validation uses only pre-cutoff data to assign segments.
    baseline, base_thresholds = user_snapshot(s, epoch('2013-03-01'), epoch('2013-03-08'))
    future = s[(s.timestamp >= epoch('2013-03-08')) & (s.timestamp < epoch('2013-03-12'))]
    baseline['return_in_next_4_days'] = baseline.index.isin(future.user_id.unique())
    bg = baseline.groupby('segment').agg(baseline_users=('R', 'size'),
        returned_users=('return_in_next_4_days', 'sum')).reindex(SEGMENTS)
    bg['subsequent_4day_return_pct'] = bg.returned_users/bg.baseline_users*100
    bg.to_csv(ROOT/'forward_validation.csv', encoding='utf-8-sig')
    # Cluster the same continuous R/F/M variables for a fair comparison.
    logx = np.log1p(u[['R', 'F', 'M']].to_numpy(dtype=float))
    x = (logx-logx.mean(axis=0))/logx.std(axis=0)
    rng = np.random.default_rng(20261012)
    indices = rng.choice(len(x), min(2000, len(x)), replace=False)
    candidates = []
    for k in [3, 4, 5]:
        inertia, labels, centers, iterations = kmeans(x, k, 20261012+k)
        sil = silhouette(x, labels, indices)
        candidates.append({'k': k, 'inertia': inertia, 'silhouette_2000': sil,
                           'iterations_best_start': iterations, 'labels': labels})
        print(f'K={k}: silhouette={sil:.4f}, inertia={inertia:.1f}', flush=True)
    best = max(candidates, key=lambda q: q['silhouette_2000'])
    # Stable display IDs sort cluster medians by F, then R.
    temp = u[['R', 'F', 'M']].copy()
    temp['cluster'] = best['labels']
    cluster_order = temp.groupby('cluster').agg(F=('F', 'median'), R=('R', 'median'))
    .sort_values(['F', 'R']).index
    mapping = {old: 'C'+str(i+1) for i, old in enumerate(cluster_order)}
    u['cluster'] = pd.Series(best['labels'], index=u.index).map(mapping)
    cg = u.groupby('cluster').agg(users=('R', 'size'), exposures=('M', 'sum'),
        R_median=('R', 'median'), F_median=('F', 'median'), M_median=('M', 'median'),
        weekly_sustained=('week_sustained', 'sum'))
    cg['user_pct'] = cg.users/len(u)*100
    cg['exposure_pct'] = cg.exposures/u.M.sum()*100
    cg.to_csv(ROOT/'cluster_summary.csv', encoding='utf-8-sig')
    cross = pd.crosstab(u.segment, u.cluster).reindex(SEGMENTS)
    cross.to_csv(ROOT/'segment_cluster_counts.csv', encoding='utf-8-sig')
```

APPENDIX B / SOURCE CODE 06

전체 파이썬 코드 · 계속

analyze_rfm.py | 원본 258~280행 | 줄 번호가 없는 점 표시는 긴 줄의 이어지는 부분입니다.

```
258     cross.div(cross.sum(axis=1), axis=0).mul(100).to_csv(ROOT/'segment_cluster_row_pct.csv',
    .   encoding='utf-8-sig')
259     assert abs(ari(u.segment.to_numpy(), u.segment.to_numpy())-1) < 1e-12
260     assert int(cross.to_numpy().sum()) == len(u)
261     u.to_csv(ROOT/'user_rfm.csv', encoding='utf-8-sig')
262     result = {'quality': meta, 'reference_utc': '2013-03-13 00:00:00+00:00',
263         'thresholds': thresholds, 'baseline_thresholds': base_thresholds,
264         'score_distribution': {col: u[col].value_counts().sort_index().to_dict()
265             for col in ['R_score', 'F_score', 'M_score']},
266         'RFM_F_M_pearson': float(u[['F', 'M']].corr().iloc[0, 1]),
267         'total_exposures': int(u.M.sum()),
268         'weekly_active': int((u.week_active_days >= 1).sum()),
269         'weekly_sustained': int(u.week_sustained.sum()),
270         'next_day_return_pct': float(u.d1_return.sum()/u.d1_eligible.sum()*100),
271         'kmeans_candidates': [{key: val for key, val in q.items() if key != 'labels'} for q in
    .   candidates],
272         'selected_k': best['k'], 'adjusted_rand_index': ari(u.segment.to_numpy(),
    .   u.cluster.to_numpy()),
273         'segment_summary': json.loads(g.reset_index().to_json(orient='records')),
274         'cluster_summary': json.loads(cg.reset_index().to_json(orient='records')),
275         'forward_validation': json.loads(bg.reset_index().to_json(orient='records'))}
276     (ROOT/'results.json').write_text(json.dumps(result, ensure_ascii=False, indent=2),
    .   encoding='utf-8')
277     print(json.dumps(result, ensure_ascii=False, indent=2), flush=True)
278
279 if __name__ == '__main__':
280     main()
```